



Release Privacy Guardian Agent

Professional Documentation PDF - Version 0.3.9

A local pre-release privacy and security scanner for public software distribution.

Document type	Program documentation, workflow explanation, screenshot analysis, safety notes
Developer credit	Sonat Birdane
Analyzed materials	Uploaded UI screenshots, FUNCTIONS, USER_GUIDE, PRIVACY_POLICY, SECURITY_NOTES, LIABILITY_AND_LIMITATIONS, RELEASE_CHECKLIST
Demonstration dataset	Six dummy TXT files containing fictional local paths, contacts, API keys, secrets, logs, installer notes, and mixed-language sensitive-looking data

Important notice

This application is a practical release-safety helper. It reduces the chance of publishing private paths, secrets, credentials, customer data, or internal release artifacts, but it does not guarantee that a project is safe, compliant, legally ready, or free from all exposure risks.

Prepared from the supplied screenshots and text documents.

Table of Contents

1. Executive Overview
2. Screenshot Analysis
3. Interface and Workflow
4. Scan Profiles and Scan Options
5. Detection Coverage
6. Severity, Release Status, and Evidence Review
7. Cleanup, Backup, and Report Handling
8. Privacy, Security, and Limitations
9. Recommended Public Release Checklist
10. Demonstration Test Case Summary
- Appendix A. Suggested Public Description

1. Executive Overview

Release Privacy Guardian Agent is designed as a local safety gate before publishing source code, desktop applications, installer packages, portable ZIP releases, documentation, screenshots, GitHub assets, or client-facing software. It scans selected folders and files for privacy, security, customer-data, local-environment, and release-hygiene risks.

The program is especially useful for independent developers and small teams who need a repeatable pre-release check before making a project public. It focuses on practical exposure patterns: local user paths, developer machine traces, secrets, credentials, database URLs, private keys, logs, reports, customer identifiers, business records, installer mistakes, backup files, and false-positive review workflows.

Core purpose	Reduce public-release mistakes by identifying sensitive or internal material before distribution.
Execution model	Local scan. The workflow is intended to run on the user computer without uploading project files.
Main release decisions	PASSED, WARNING, or BLOCKED based on selected scan profile, options, and finding severity.
Primary risk classes	Developer privacy, local paths, secrets, credentials, customer/client data, debug traces, installer/build artifacts, reports, and risky file types.
Safety philosophy	Use the tool as a pre-release safety layer, then perform a manual final review.

Documentation conclusion

The supplied screenshots demonstrate a complete workflow: clean launch state, scan execution, BLOCKED release decision, detailed finding review, evidence redaction, cleanup behavior, and backup creation.

2. Screenshot Analysis

2.1 Main Interface Before Scan

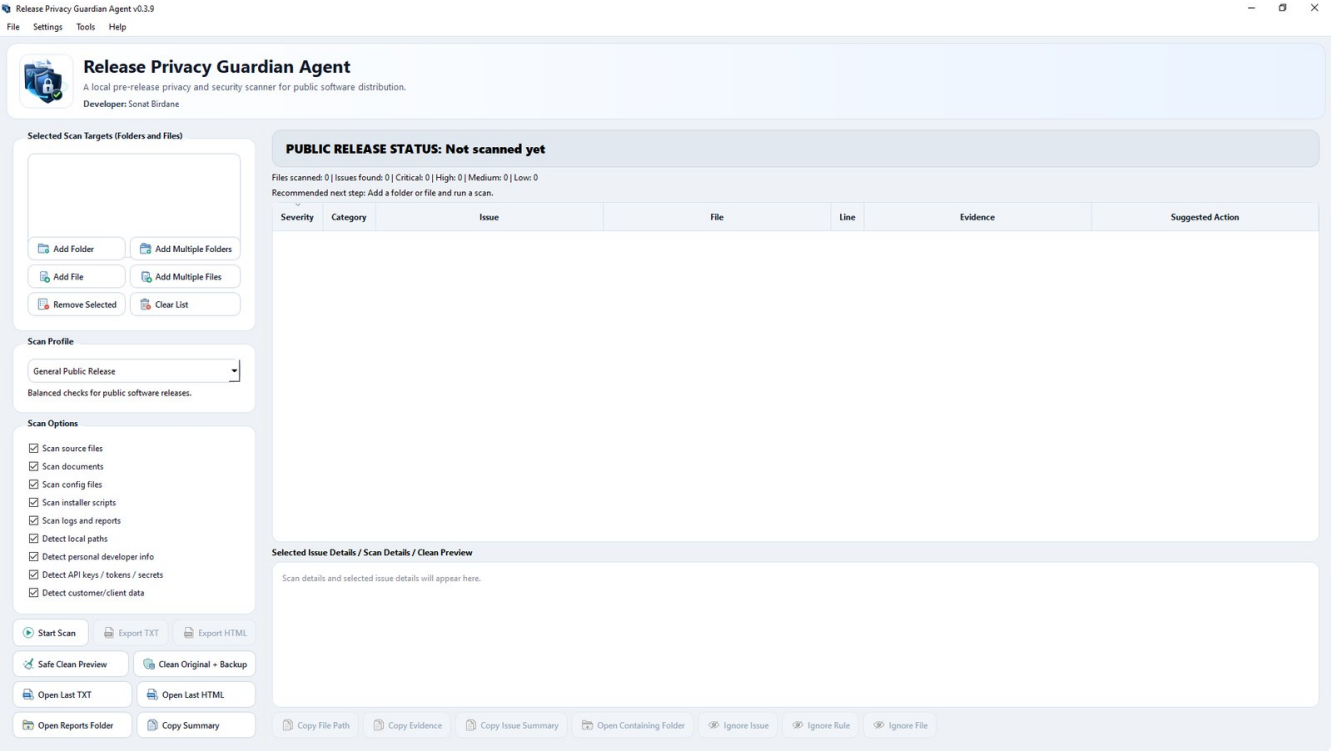


Figure 1 - Initial GUI state: selected target area, scan profile, scan options, issue table, details panel, and report/cleanup action buttons.

The first screenshot shows the application in a clean pre-scan state. The left panel is organized around target selection, scan profile selection, and broad scan options. The central panel is reserved for release status and the issue table. The lower panel is prepared for selected issue details, scan details, and clean preview output.

A key UX choice is visible here: report export, clean, copy, open, and ignore actions remain contextual. The user is guided to add a folder or file before running a scan, reducing the chance of exporting or modifying an empty or irrelevant review state.

2.2 BLOCKED Scan Result

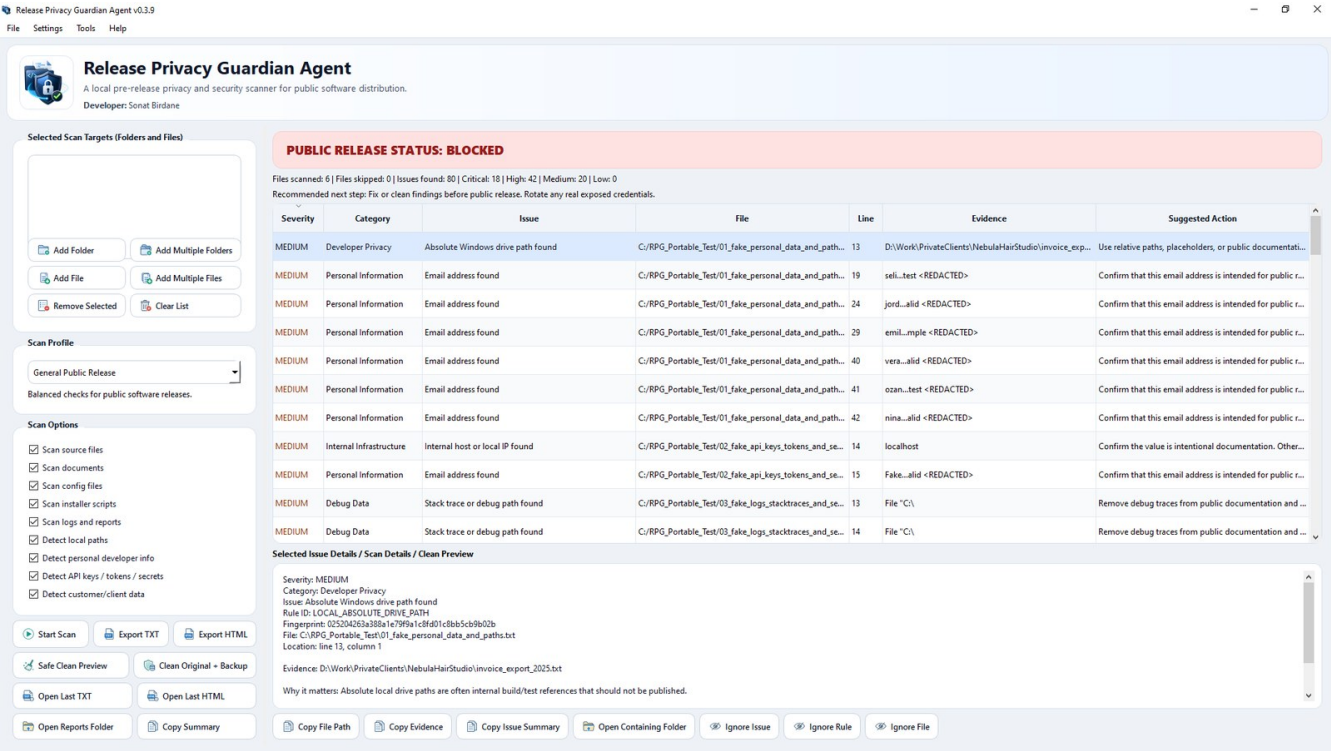


Figure 2 - After scanning the dummy TXT dataset, the release status becomes BLOCKED and the issue table lists detected findings.

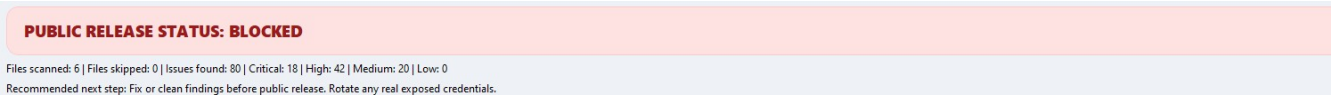


Figure 2A - Status banner and scan counters: 6 files scanned, 80 issues found, 18 Critical, 42 High, 20 Medium, 0 Low.

The second screenshot demonstrates the release-decision behavior. The dummy dataset contains deliberately suspicious content, so the program marks the package as BLOCKED. The summary shows 6 scanned files, 0 skipped files, 80 total issues, 18 critical findings, 42 high findings, 20 medium findings, and 0 low findings. The issue table separates severity, category, issue description, file, line, evidence, and suggested action. This makes the scan result usable as an actionable review queue rather than a simple text dump.

2.3 Finding Details, Evidence, and Suggested Action

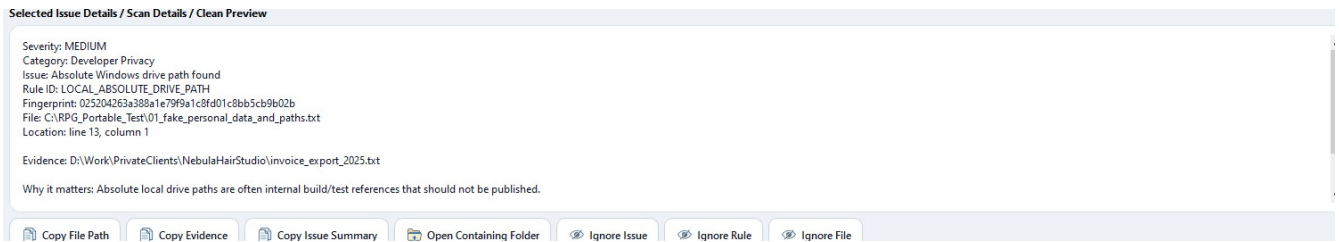


Figure 2B - Selected issue details show severity, category, rule ID, fingerprint, file, location, evidence, why it matters, and suggested action.



Figure 3 - Detailed report examples: local path, address field, possible AWS secret assignment, database connection string, private key block, JWT token, and generic secret assignment detections.

The details area is one of the program's strongest documentation points. It shows not only that a pattern was found, but also why it matters and what the user should do next. Evidence is partially redacted where appropriate, which helps users review issues without unnecessarily expanding secret exposure in the interface or reports.

The screenshots also show that the tool handles multiple risk classes in one pass: developer privacy paths, customer/personal fields, API secrets, private key blocks, JWT-like tokens, database connection strings, stack trace paths, and generic credential assignments.

2.4 Clean Original + Backup Demonstration

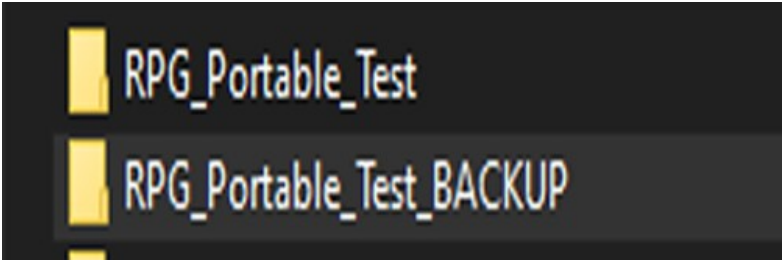


Figure 4A - Backup folder created beside the original demo folder.

RPG DUMMY TEST FILE 01 Purpose: fake personal/developer information and local paths. Everything in this file is fictional and safe for testing only. Developer Notes: Developer: Alican Demo Backup owner: Mira Testova Internal reviewer: Can Fakeperson Local Development Paths: C:\Users\AlicanDemo\Desktop\ReleaseBuild\client_private_notes.txt C:\Users\MiraTestova\Downloads\temporary_build\secret_old_config.ini D:\Work\PrivateClients\NebulaHairStudio\invoice_export_2025.txt /Users/alican.demo/Documents/UnityProjects/PrivatePrototype/user_export.txt /home/mira/projects/release_candidate/.env.backup Fake Contact List: Name: Selin Example Email: selin.example.demo@fake-mail.test Phone: +90 555 010 22 33 Address: 19 Demo Street, Sample District, Izmir, 35000 Name: Jordan Placeholder Email: jordan.placeholder@demo-company.invalid Phone: +1 202 555 0147 Address: 404 Fictional Avenue, Test City, CA 90000 Name: Emily Sandbox Email: emily.sandbox@dummy-client.example Phone: +44 7700 900123 Address: 88 Placeholder Road, London, AB1 2CD	RPG DUMMY TEST FILE 01 Purpose: fake personal/developer information and local paths. Everything in this file is fictional and safe for testing only. Developer Notes: Developer: Alican Demo Backup owner: Mira Testova Internal reviewer: Can Fakeperson Local Development Paths: example/project-path example/project-path example/project-path example/project-path example/project-path Fake Contact List: Name: Selin Example Email: example@example.com Phone: example-phone Address: example address Name: Jordan Placeholder Email: example@example.com Phone: example-phone Address: example address Name: Emily Sandbox Email: example@example.com Phone: example-phone Address: example address Internal Reminder:
--	---

Figure 4B - Before/after cleanup view: local paths, emails, phones, and addresses are replaced with public-friendly example values.

The clean workflow demonstrated in the screenshot keeps the original release target and creates a sibling backup folder. The before/after text comparison shows that risky content is replaced with public-friendly values such as example/project-path, example@example.com, example-phone, and example address.

This behavior is more useful than blank deletion for documentation and public examples: it preserves the structure and readability of the file while removing sensitive-looking material. Users still need to re-scan and manually inspect the cleaned result before release.

3. Interface and Workflow

3.1 Target Selection

The GUI supports both folder and file based review. A user can scan one folder recursively, multiple folders, one direct file, or multiple direct files. Mixed target lists are useful when a release package includes both a project folder and external documentation, installer scripts, or final asset manifests.

Control	Function
Add Folder	Adds one folder for recursive scanning.
Add Multiple Folders	Adds multiple folders for recursive scanning.
Add File	Adds one selected file directly.
Add Multiple Files	Adds multiple selected files directly.
Remove Selected / Clear List	Removes one target or clears the scan queue before running another review.

3.2 Scan Execution and Review Flow

- Select the release candidate folders and/or files.
- Choose the scan profile that matches the target project type.
- Enable or disable scan categories based on the review objective.
- Click Start Scan and review the PASSED, WARNING, or BLOCKED decision.
- Sort and inspect the issue table, beginning with CRITICAL and HIGH findings.
- Use the details panel to understand the rule, evidence, suggested action, and review note.
- Export reports only after checking that the report itself is safe to share.
- Use Safe Clean Preview before any cleanup operation.
- Use Clean Original + Backup only after reviewing the preview and understanding the backup behavior.
- Re-scan the cleaned output and manually inspect important files before publication.

4. Scan Profiles and Scan Options

Scan profiles tune the review emphasis for different types of public release candidates. A strict profile is intentionally noisier and should be used when the user wants maximum sensitivity before publication.

Profile	Best use case
General Public Release	Balanced default mode for public software releases.
Python Desktop App	Source code, logs, package artifacts, config files, PyInstaller-style output, and desktop app release folders.
Installer Package	Installer scripts, batch/PowerShell files, release folders, setup assets, and package hygiene.
Business / Client Software	Customer, phone, address, invoice, tax, order, appointment, and business data oriented review.
Game / 3D Pipeline Tool	Maya, Autodesk, Unreal, Unity, FBX, texture, asset, content, and local pipeline path patterns.
Strict Privacy Audit	Most aggressive mode for final pre-publication review; more false positives are expected.

The visible scan options allow the user to adjust broad coverage categories. In the supplied screenshot all major options are enabled, including source files, documents, config files, installer scripts, logs/reports, local paths, personal developer info, API keys/tokens/secrets, and customer/client data.

5. Detection Coverage

The program uses pattern-based detection to highlight practical public-release risks. The screenshots show a representative subset of detections, while the supplied function documentation describes a broader rule set.

Category	Examples of covered patterns
Developer privacy and local environment	Windows user paths, Unix/macOS home paths, absolute drive paths, local machine references, debug path traces.
Secrets and credentials	Private key blocks, generic secret assignments, AWS keys, GitHub/GitLab tokens, Slack/Discord tokens or webhooks, Stripe keys, database URLs, JWT-like tokens, Bearer tokens, cookie/session/auth values.
Customer and business data	Names, email addresses, phone numbers, addresses, billing/shipping/delivery data, tax/VAT numbers, order IDs, appointment identifiers, customer export file names.
Release hygiene	.env files, database files, logs, reports, crash dumps, temporary files, backup files, installer build traces, customer export artifacts.
Multilingual data labels	English and Turkish business/customer terms, plus support for several additional language field labels.

Pattern-based, not magical

The scanner can catch many common mistakes quickly, but it cannot guarantee detection of encoded, split, compressed, encrypted, obfuscated, image-based, binary, or unusually formatted sensitive data.

6. Severity, Release Status, and Evidence Review

6.1 Severity Model

Severity	Meaning in release review
LOW	Minor release-hygiene issue.
MEDIUM	Internal configuration, infrastructure, or possible exposure that should be reviewed.
HIGH	Developer privacy, customer data, local path, personal email, or significant release risk.
CRITICAL	Secret, token, private key, credential, or comparable release blocker.

6.2 Release Status Logic

Status	Interpretation
PASSED	No blocking issues were found based on the selected rules and options.
WARNING	Non-blocking issues were found; review before release.
BLOCKED	High or Critical issues were found. Public release should not proceed until reviewed and resolved.

6.3 Evidence Handling

Each issue row contains direct evidence and a suggested action. The selected issue panel expands the finding with rule ID, fingerprint, file path, line/column, evidence, reason, suggested action, and review note. In the screenshot examples, sensitive-looking evidence is partially redacted, such as token fragments and email content.

This combination supports triage. Users can prioritize CRITICAL and HIGH issues, then review MEDIUM items for context. Findings that are intentional and safe can be handled through ignore actions or whitelist entries, but those features should not be used simply to force a PASSED state.

7. Cleanup, Backup, and Report Handling

7.1 Safe Clean Preview

Safe Clean Preview is a non-destructive review step. It shows planned replacements and removals before files are modified. This step should be treated as mandatory before using Clean Original + Backup.

7.2 Clean Original + Backup

Clean Original + Backup creates a backup first, then modifies the selected original target in place. For folder targets, the backup is created as a sibling BACKUP folder. For direct file targets, backups are created outside the release folder under ParentFolder_BACKUP/ONLYFILES to reduce accidental publication of backup copies.

Target type	Original	Backup behavior
Folder target	<PROJECT_FOLDER>	<PROJECT_FOLDER>_BACKUP
Direct file target	selected-file config.txt	ParentFolder_BACKUP/ ONLYFILES/config.txt

7.3 Public-Friendly Replacement Values

The cleanup examples use readable public placeholders rather than harsh deletion markers. This keeps documentation and sample files understandable after cleaning.

example, example@example.com, example/project-path, example/downloads, example-machine, example.local, example customer, example business, example company, example address, example-phone, example-tax-number, example-id-number, example-invoice-id, example-order-id, example public note, example-postal-code, example-bank-account, example-date

7.4 Report Export

TXT and HTML reports can be exported after scan results exist. Reports may still contain local context, file paths, issue summaries, and redacted evidence. They should be reviewed before being shared publicly.

8. Privacy, Security, and Limitations

8.1 Local-Only Operation

The privacy model presented in the uploaded documents positions the application as a local scanner: project files are scanned on the user computer, scan results are not transmitted to a remote server, and scanning does not require an internet connection.

8.2 Local Settings and Artifacts

The application may store local settings under the user profile, including selected targets, scan profile, scan options, whitelist phrases, ignored findings, recent report paths, and window size. Reports and backups are created only as local artifacts controlled by the user.

8.3 Single-Developer and No-Guarantee Notice

The documentation explicitly frames the program as a single-developer project. Bugs, missed detections, false positives, false negatives, packaging mistakes, and edge-case failures remain possible. The tool reduces risk; it does not transfer final release responsibility away from the user.

8.4 What Users Must Still Do

- Review every HIGH and CRITICAL finding, then review MEDIUM findings with context.
- Rotate or revoke any real exposed credential. Replacing it in files is not enough.
- Inspect reports before sharing them publicly.
- Confirm backup folders are not included in public ZIPs, installers, GitHub release assets, or screenshots.
- Open important cleaned files manually and confirm that structure and meaning are still correct.
- Perform a final human review of source tree, release assets, installer output, portable ZIP, documentation, screenshots, and release notes.

9. Recommended Public Release Checklist

This condensed checklist converts the supplied release checklist into a practical user-facing workflow for the PDF documentation.

1. Select the correct project folder(s) and/or file(s).
2. Run General Public Release scan.
3. Run Strict Privacy Audit scan.
4. Review every HIGH and CRITICAL finding.
5. Review MEDIUM findings, especially internal hosts, local paths, and business/customer data.
6. Rotate or revoke any real credential that appears in a finding or release artifact.
7. Export TXT/HTML reports only if needed, then inspect the exported reports before sharing.
8. Use Safe Clean Preview before any cleanup action.
9. Use Clean Original + Backup only after reviewing the preview.
10. Confirm backup creation succeeded and that backup folders are outside the public release package.
11. Re-scan the cleaned original target.
12. Manually open important cleaned files and inspect the result.
13. Confirm no .env, database, log, backup, temp, crash dump, customer export, or test file is included accidentally.
14. Review installer scripts, installer output, portable ZIP contents, GitHub release assets, documentation, and screenshots.
15. Publish only after a final manual review.

10. Demonstration Test Case Summary

The screenshots were produced by scanning six dummy TXT files containing fictional data. The content was intentionally designed to be noisy and challenging so the program could demonstrate realistic detection behavior without exposing real private information.

Dummy file theme	Representative risks tested
Fake personal data and paths	Developer names, local paths, contacts, customer export rows.
Fake API keys, tokens, and secrets	OpenAI-like keys, Stripe-like keys, GitHub tokens, AWS examples, database URL, JWT secret, fake private key blocks.
Fake logs and stack traces	Debug paths, session IDs, CSRF token, cookies, traceback, localhost and internal URLs.
Fake installer config and build notes	Build machine paths, signing certificate path, signing password, installer output path, accidental internal-roadmap inclusion.
Mixed language sensitive content	Turkish and English local paths, phone/email, password/token, card-like and IBAN-like samples.
False-positive stress	Repeated paths, secret label variants, documentation sentences using words such as password, token, secret, and key.

Screenshot result

The demonstration scan produced a **BLOCKED** status with 80 issues across 6 files. This is expected for the dummy dataset and confirms that the scanner surfaces multiple risk classes in a single review workflow.

Appendix A. Suggested Public Description

The following text can be used or adapted for README, Gumroad, GitHub, or LinkedIn documentation.

Release Privacy Guardian Agent is a local pre-release privacy and security scanner for developers preparing public software distributions. It reviews selected folders and files for sensitive release risks such as local developer paths, secrets, tokens, private keys, database connection strings, customer/client data, debug traces, installer artifacts, logs, reports, and risky backup files. The tool provides a clear PASSED, WARNING, or BLOCKED release status, detailed findings with evidence and suggested actions, TXT/HTML report export, false-positive controls, Safe Clean Preview, and Clean Original + Backup cleanup support. It is intended as a practical safety layer before publishing GitHub projects, installer packages, portable ZIP files, documentation, screenshots, and client-facing tools. Final manual review remains required before every public release.

Appendix B. Practical Screenshot Guidance

For public screenshots, use a neutral demo folder such as C:/RPG_Portable_Test or C:/PublicReleaseTest. Avoid selecting folders under C:/Users/<name>/Desktop or Downloads when preparing marketing, README, or documentation images. Even when the test data is fake, the screenshot should avoid revealing personal machine paths.

Final reminder

Release Privacy Guardian Agent can reduce release risk, but users remain responsible for deciding what is safe to publish.